

UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

Facultad de Ingeniería y Ciencias Básicas

INFORME DE LABORATORIO

Aprovisionamiento de máquinas virtuales con Vagrant

Asignatura

Computación en la Nube

Docente

Heberth Fabián Martínez Velásquez

Presentado por

Santiago Bonilla Idrobo

Santiago de Cali

Noviembre de 2026

Contenido

1. Introducción *3*

2. Objetivos *3*

2.1 Objetivo general 3

2.2 Objetivos específicos 3

3. Marco teórico *3*

3.1 Virtualización y modelos de servicio 3

3.2 Vagrant y VirtualBox 4

3.3 Arquitecturas x86_64 y ARM64 4

4. Entorno de trabajo *4*

5. Desarrollo del laboratorio *5*

5.1 Configuración inicial del Vagrantfile 5

5.2 Incidencia 1: arquitectura x86 no soportada en ARM 5

5.3 Incidencia 2: box bento/centos-stream-9 sin versión ARM64 6

5.4 Vagrantfile ajustado 6

5.5 Despliegue de las máquinas virtuales 7

5.6 Verificación de acceso y conectividad 9

5.7 Empaquetado de la box 10

5.8 Intento de publicación en HCP Vagrant Registry 11

6. Análisis de resultados *12*

7. Conclusiones *13*

8. Referencias *13*

9. Directorios sincronizados en Vagrant *14*

1. Introducción

La computación en la nube se apoya en la virtualización para ofrecer recursos de cómputo bajo demanda, elásticos y medidos. Herramientas como Vagrant permiten describir esos recursos como código: un único archivo (Vagrantfile) define qué máquinas virtuales se crean, con qué sistema operativo, qué recursos y qué configuración de red, de forma reproducible en cualquier equipo.

En este laboratorio se aprovisionó un entorno de dos máquinas virtuales (cliente y servidor) con Vagrant y VirtualBox, se empaquetó una de ellas como box reutilizable y se publicó para su distribución. El trabajo se realizó sobre un Mac mini con procesador Apple M4 (arquitectura ARM64), lo que obligó a adaptar la guía original, diseñada para equipos x86_64, y a resolver dos cambios recientes del ecosistema: la descontinuación de CentOS 8 y el cierre de la creación de boxes en HCP Vagrant Registry (antes Vagrant Cloud).

2. Objetivos

2.1 Objetivo general

Aprovisionar, empaquetar y publicar máquinas virtuales mediante Vagrant, aplicando los conceptos de virtualización e infraestructura como servicio vistos en la asignatura.

2.2 Objetivos específicos

- Definir un entorno multimáquina (cliente y servidor) en un Vagrantfile con red privada entre ambos nodos.
- Identificar y resolver las incompatibilidades de arquitectura entre boxes x86_64 y un anfitrión ARM64.
- Verificar el funcionamiento y la conectividad de las máquinas desplegadas.
- Empaquetar una máquina como box y publicarla con versionado para que pueda ser consumida por otros usuarios.

3. Marco teórico

3.1 Virtualización y modelos de servicio

La virtualización permite ejecutar varias máquinas virtuales aisladas sobre un mismo hardware físico mediante un hipervisor. Es la base del modelo IaaS (Infrastructure as a Service), en el que, según el NIST (SP 800-145 y SP 800-146), el proveedor mantiene el control administrativo del hardware y del hipervisor, mientras el consumidor controla el sistema operativo invitado y las aplicaciones. En este laboratorio el propio estudiante cumple ambos roles a escala local: VirtualBox actúa como hipervisor y Vagrant como la capa de gestión que recibe las peticiones del usuario.

3.2 Vagrant y VirtualBox

Vagrant es una herramienta de HashiCorp para crear y gestionar entornos virtuales a partir de un archivo declarativo. Delega la ejecución en un provider (VirtualBox, VMware, QEMU, entre otros). Una box es una imagen base empaquetada que incluye el disco del sistema operativo y metadatos como el provider y la arquitectura para los que fue construida.

3.3 Arquitecturas x86_64 y ARM64

Los procesadores Apple Silicon (M1 a M4) usan el conjunto de instrucciones ARM64. VirtualBox en macOS sobre Apple Silicon solo ejecuta máquinas virtuales ARM64 y no emula x86_64, por lo que una box construida para x86_64 no puede arrancar en ese anfitrión, aunque la configuración del Vagrantfile sea correcta.

4. Entorno de trabajo

Tabla 1. *Especificaciones del entorno*

Componente	Detalle
Equipo anfitrión	Mac mini, chip Apple M4 (ARM64)
Sistema operativo anfitrión	macOS 27.2
Hipervisor / provider	Oracle VirtualBox 7.2.2
Gestor de entornos	Vagrant 2.4.9
Box utilizada	bento/centos-stream-9, arquitectura arm64
Máquinas virtuales	servidor (192.168.100.3) y cliente (192.168.100.2)

La versión de VirtualBox y de Vagrant puede consultarse con los siguientes comandos:

```
VBoxManage --version
vagrant --version
```

```
You're running an up-to-date version of Vagrant!
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % VBoxManage --version
7.2.22r175245
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas %
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % vagrant version
Installed Version: 2.4.9
Latest Version: 2.4.9

You're running an up-to-date version of Vagrant!
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas %
```

Figura 1. *versiones de VirtualBox y Vagrant instaladas en el equipo*

5. Desarrollo del laboratorio

5.1 Configuración inicial del Vagrantfile

Siguiendo la guía de la asignatura, se creó un directorio de trabajo y un Vagrantfile con dos máquinas, servidor y cliente, basadas en la box bento/centos-stream-9 .

```
config.vm.define :servidor do |servidor|
  servidor.vm.box = " bento/centos-stream-9 "
  ...
end
```

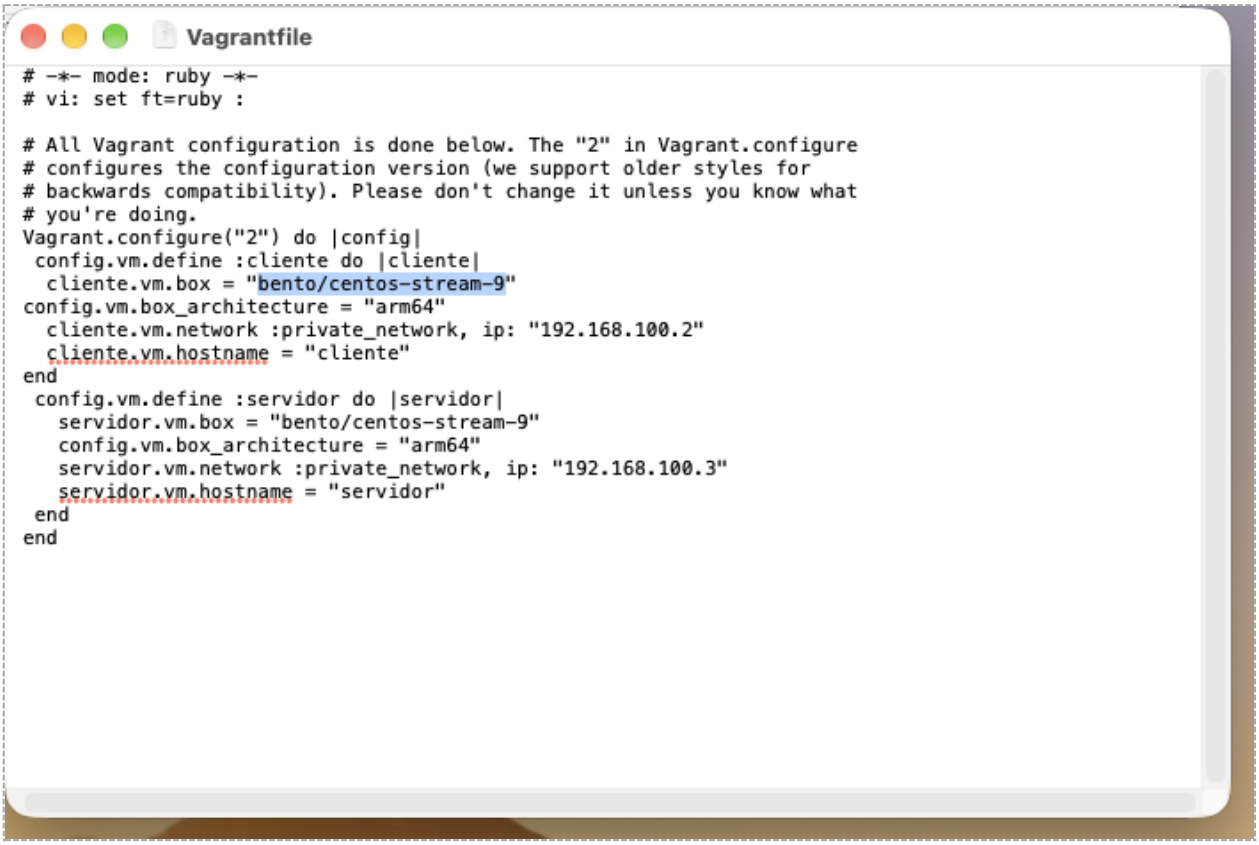


Figura 2. *Vagrantfile inicial con las máquinas servidor y cliente sobre bento/centos-stream-9*

5.2 Incidencia 1: arquitectura x86 no soportada en ARM

Al ejecutar vagrant up, VirtualBox rechazó el arranque de la máquina con el siguiente error:

```
VBoxManage: error: Cannot run the machine because its platform
architecture x86 is not supported on ARM
VBoxManage: error: Details: code VBOX_E_PLATFORM_ARCH_NOT_SUPPORTED
```

El análisis del error mostró que la box descargada había sido construida para x86_64, mientras que el anfitrión es ARM64. Como VirtualBox en Apple Silicon no emula x86, la única solución es

usar una box construida para ARM64 (o cambiar a un provider con emulación, como QEMU, con una penalización importante de rendimiento). Antes de continuar se eliminó la máquina creada parcialmente:

```
vagrant destroy -f
```



Figura 3. *error VBOX_E_PLATFORM_ARCH_NOT_SUPPORTED al ejecutar vagrant up*

5.3 Incidencia 2: box bento/centos-stream-9 sin versión ARM64

Al indicar explícitamente la arquitectura arm64 en el Vagrantfile (box_architecture), Vagrant informó que la box centos/8 no tiene variante para esa arquitectura:

```
The box you're attempting to add doesn't support the requested
architecture. Please find an alternate box that support the
requested architecture.

Box: centos/8
Architecture: arm64
```

5.4 Vagrantfile ajustado

Se reescribió el Vagrantfile con la box bento/centos-stream-9 para arm64, definiendo la box y la arquitectura dentro del bloque de cada máquina, una red privada en el rango 192.168.56.0/24 (el permitido por defecto por VirtualBox en macOS) y desactivando la carpeta sincronizada, que presenta fallos con las Guest Additions en ARM.

```
BOX = "bento/centos-stream-9 "
ARCH = "arm64"

Vagrant.configure("2") do |config|
  config.vm.synced_folder ".", "/vagrant", disabled: true

  config.vm.provider "virtualbox" do |vb|
    vb.memory = 1024
    vb.cpus = 1
  end

  config.vm.define :servidor do |servidor|
```

```
    servidor.vm.box = BOX
    servidor.vm.box_architecture = ARCH
    servidor.vm.hostname = "servidor"
    servidor.vm.network "private_network", ip: "192.168.56.10"
  end

  config.vm.define :cliente do |cliente|
    cliente.vm.box = BOX
    cliente.vm.box_architecture = ARCH
    cliente.vm.hostname = "cliente"
    cliente.vm.network "private_network", ip: "192.168.56.11"
  end
end
```

[Insertar captura: Vagrantfile final con Rocky Linux 8 arm64]

Figura 5. *Vagrantfile final con Rocky Linux 8 arm64*

5.5 Despliegue de las máquinas virtuales

Se descargó la box para la arquitectura correcta y se levantaron ambas máquinas:

```
vagrant box add bento/centos-stream-9  --provider virtualbox --architecture arm64
vagrant up
vagrant status
```

El comando vagrant status confirmó que las máquinas servidor y cliente quedaron en estado running con el provider VirtualBox.

```
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % vagrant up
Bringing machine 'cliente' up with 'virtualbox' provider...
Bringing machine 'servidor' up with 'virtualbox' provider...
==> cliente: Importing base box 'bento/centos-stream-9'...
==> cliente: Matching MAC address for NAT networking...
==> cliente: Checking if box 'bento/centos-stream-9' version '202510.26.0' is up to date...
==> cliente: Setting the name of the VM: Maquinas_cliente_1794265936574_76464
==> cliente: Fixed port collision for 22 => 2222. Now on port 2250.
Vagrant is currently configured to create VirtualBox synced folders with
the `SharedFoldersEnableSymlinksCreate` option enabled. If the Vagrant
guest is not trusted, you may want to disable this option. For more
information on this option, please refer to the VirtualBox manual:

  https://www.virtualbox.org/manual/ch04.html#sharedfolders

This option can be disabled globally with an environment variable:

  VAGRANT_DISABLE_VBOXSYMLINKCREATE=1

or on a per folder basis within the Vagrantfile:

  config.vm.synced_folder '/host/path', '/guest/path', SharedFoldersEnableSymlinksCreate: false
==> cliente: Clearing any previously set network interfaces...
==> cliente: Preparing network interfaces based on configuration...
    cliente: Adapter 1: nat
    cliente: Adapter 2: hostonly
==> cliente: Forwarding ports...
    cliente: 22 (guest) => 2250 (host) (adapter 1)
==> cliente: Booting VM...
==> cliente: Waiting for machine to boot. This may take a few minutes...
    cliente: SSH address: 127.0.0.1:2250
    cliente: SSH username: vagrant
    cliente: SSH auth method: private key
    cliente:
    cliente: Vagrant insecure key detected. Vagrant will automatically replace
    cliente: this with a newly generated keypair for better security.
    cliente:
    cliente: Inserting generated public key within guest...
    cliente: Removing insecure key from the guest if it's present...
    cliente: Key inserted! Disconnecting and reconnecting using new SSH key...
==> cliente: Machine booted and ready!
==> cliente: Checking for guest additions in VM...
==> cliente: Setting hostname...
==> cliente: Configuring and enabling network interfaces...
==> cliente: Mounting shared folders...
    cliente: /Users/santiagobonillaidrobo/Documents/CURSO_IA/CursoUAO/Computacion en la nube/Maquinas => /vagrant
==> servidor: Importing base box 'bento/centos-stream-9'...
==> servidor: Matching MAC address for NAT networking...
==> servidor: Checking if box 'bento/centos-stream-9' version '202510.26.0' is up to date...
==> servidor: Setting the name of the VM: Maquinas_servidor_1794266024600_13660
==> servidor: Fixed port collision for 22 => 2222. Now on port 2249.
==> servidor: Clearing any previously set network interfaces...
==> servidor: Preparing network interfaces based on configuration...
    servidor: Adapter 1: nat
    servidor: Adapter 2: hostonly
==> servidor: Forwarding ports...
    servidor: 22 (guest) => 2249 (host) (adapter 1)
==> servidor: Booting VM...
==> servidor: Waiting for machine to boot. This may take a few minutes...
    servidor: SSH address: 127.0.0.1:2249
    servidor: SSH username: vagrant
    servidor: SSH auth method: private key
    servidor:
    servidor: Vagrant insecure key detected. Vagrant will automatically replace
    servidor: this with a newly generated keypair for better security.
    servidor:
    servidor: Inserting generated public key within guest...
    servidor: Removing insecure key from the guest if it's present...
    servidor: Key inserted! Disconnecting and reconnecting using new SSH key...
==> servidor: Machine booted and ready!
==> servidor: Checking for guest additions in VM...
==> servidor: Setting hostname...
==> servidor: Configuring and enabling network interfaces...
==> servidor: Mounting shared folders...
    servidor: /Users/santiagobonillaidrobo/Documents/CURSO_IA/CursoUAO/Computacion en la nube/Maquinas => /vagrant
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % vagrant golbal-status
Usage: vagrant [options] <command> [<args>]

    -h, --help                     Print this help.

Common commands:
     autocomplete    manages autocomplete installation on host
     box             manages boxes: installation, removal, etc.
     cloud           manages everything related to Vagrant Cloud
     destroy         stops and deletes all traces of the vagrant machine
     global-status   outputs status Vagrant environments for this user
     halt            stops the vagrant machine
```

Figura 6. *salida de vagrant up con la creación de servidor y cliente*

```
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % vagrant global-status
id       name     provider   state   directory
------------------------------------------------------------------------------------------------------------------
45473d4  cliente  virtualbox running /Users/santiagobonillaidrobo/Documents/CURSO_IA/CursoUAO/Computacion en la nube/Maquinas
c8e8b9e  servidor virtualbox running /Users/santiagobonillaidrobo/Documents/CURSO_IA/CursoUAO/Computacion en la nube/Maquinas
```

Figura 7. *vagrant status con ambas máquinas en estado running*

5.6 Verificación de acceso y conectividad

Se ingresó a cada máquina por SSH y se verificó el sistema operativo, la arquitectura y la comunicación por la red privada:

```
vagrant ssh cliente
sudo -i
yum install net-tools
yum install vim
ping 192.168.100.3
```

```
[vagrant@cliente ~]$ vagrant ssh cliente
-bash: vagrant: command not found
[vagrant@cliente ~]$ sudo -i
[root@cliente ~]# yum install net-tools
Last metadata expiration check: 0:01:36 ago on Mon 09 Nov 2026 11:52:49 PM UTC.
Package net-tools-2.0-0.64.20160912git.el9.aarch64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@cliente ~]# yum install vim
Last metadata expiration check: 0:02:00 ago on Mon 09 Nov 2026 11:52:49 PM UTC.
Package vim-enhanced-2:8.2.2637-43.el9.aarch64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@cliente ~]# ifconfig
enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fd17:625c:f037:2:a00:27ff:fe80:861a  prefixlen 64  scopeid 0x0<global>
        inet6 fe80::a00:27ff:fe80:861a  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:80:86:1a  txqueuelen 1000  (Ethernet)
        RX packets 25450  bytes 25910897 (24.7 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 9932  bytes 808381 (789.4 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

enp0s9: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.100.2  netmask 255.255.255.0  broadcast 192.168.100.255
        inet6 fe80::a00:27ff:fe53:bc80  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:53:bc:80  txqueuelen 1000  (Ethernet)
        RX packets 160  bytes 13634 (13.3 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 626 (626.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figura 8. *Instalacion drivers de red y vim*

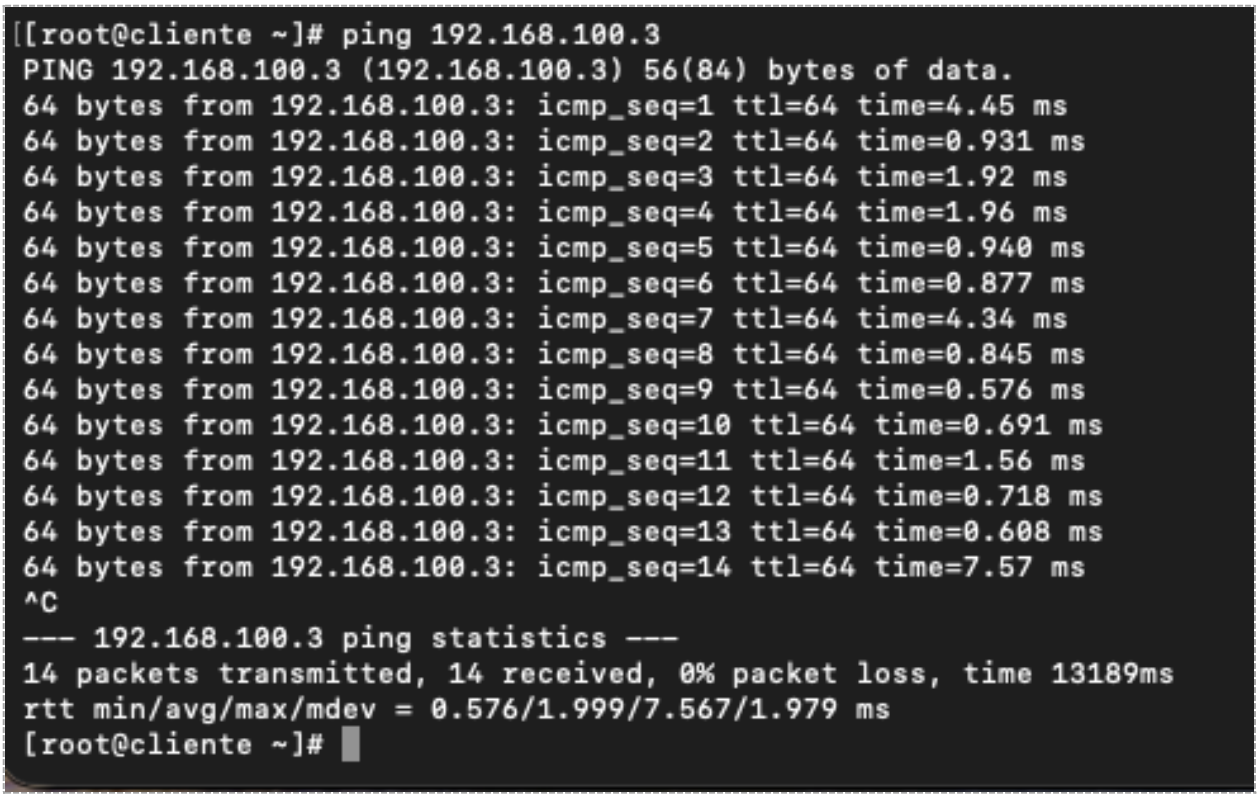


Figura 9. *prueba de ping entre servidor y cliente por la red privada*

5.7 Empaquetado de la box

Una vez validada la máquina servidor, se empaquetó como una box reutilizable y se calculó su checksum SHA-256 para garantizar la integridad del archivo al distribuirlo:

```
vagrant package servidor --output servidor.box
shasum -a 256 servidor.box
```

```
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % vagrant suspend
==> cliente: Saving VM state and suspending execution...
==> servidor: Saving VM state and suspending execution...
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % vagrant package servidor --output mynew.box
==> servidor: Discarding saved state of VM...
==> servidor: Clearing any previously set forwarded ports...
==> servidor: Exporting VM...
==> servidor: Compressing package to: /Users/santiagobonillaidrobo/Documents/CURSO_IA/CursoUAO/Computacion en la nube/Maquinas/mynew.box
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % vagrant box add mynewbox mynew.box
==> box: Box file was not detected as metadata. Adding it directly...
==> box: Adding box 'mynewbox' (v0) for provider: (arm64)
    box: Unpacking necessary files from: file:///Users/santiagobonillaidrobo/Documents/CURSO_IA/CursoUAO/Computacion%20en%20la%20nube/Maquinas/mynew.box
==> box: Successfully added box 'mynewbox' (v0) for '(arm64)'!
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % vagrant package servidor --output servidor.box
==> servidor: Exporting VM...
==> servidor: Compressing package to: /Users/santiagobonillaidrobo/Documents/CURSO_IA/CursoUAO/Computacion en la nube/Maquinas/servidor.box
santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % vagrant vertsion
Usage: vagrant [options] <command> [<args>]

    -h, --help                     Print this help.

Common commands:
     autocomplete    manages autocomplete installation on host
     box             manages boxes: installation, removal, etc.
     cloud           manages everything related to Vagrant Cloud
     destroy         stops and deletes all traces of the vagrant machine
     global-status   outputs status Vagrant environments for this user
     halt            stops the vagrant machine
     help            shows the help for a subcommand
     init            initializes a new Vagrant environment by creating a Vagrantfile
     login
     package         packages a running vagrant environment into a box
     plugin          manages plugins: install, uninstall, update, etc.
     port            displays information about guest port mappings
     powershell      connects to machine via powershell remoting
     provision       provisions the vagrant machine
     push            deploys code in this environment to a configured destination
     rdp             connects to machine via RDP
     reload          restarts vagrant machine, loads new Vagrantfile configuration
     resume          resume a suspended vagrant machine
     snapshot        manages snapshots: saving, restoring, etc.
     ssh             connects to machine via SSH
     ssh-config      outputs OpenSSH valid configuration to connect to the machine
     status          outputs status of the vagrant machine
     suspend         suspends the machine
     up              starts and provisions the vagrant environment
     upload          upload to machine via communicator
     validate        validates the Vagrantfile
     version         prints current and latest Vagrant version
     winrm           executes commands on a machine via WinRM
     winrm-config    outputs WinRM configuration to connect to the machine

For help on any individual command run `vagrant COMMAND -h`

Additional subcommands are available, but are either more advanced
or not commonly used. To see all subcommands, run the command
`vagrant list-commands`.
        --[no-]color                 Enable or disable color output
        --machine-readable           Enable machine readable output
    -v, --version                    Display Vagrant version
        --debug                      Enable debug output
        --timestamp                  Enable timestamps on log output
        --debug-timestamp            Enable debug output with timestamps
        --no-tty                     Enable non-interactive output

santiagobonillaidrobo@Mac-mini-de-Santiago Maquinas % vagrant version
Installed Version: 2.4.9
Latest Version: 2.4.9
```

Figura 10. *generación del archivo servidor.box y su checksum*

5.8 Intento de publicación en HCP Vagrant Registry

La guía indica publicar la box en app.vagrantup.com/boxes/new. Ese servicio (Vagrant Cloud) fue migrado por HashiCorp a HashiCorp Cloud Platform con el nombre de HCP Vagrant Registry. Al intentar crear el registry y la box en el portal de HCP, la plataforma devolvió el error:

```
Error
resource creation is currently disabled
```

La causa es la descontinuación del servicio anunciada por HashiCorp: desde el 1 de octubre de 2026 no es posible crear nuevas boxes ni registries en HCP Vagrant, y los usuarios existentes deben migrar sus boxes a otro proveedor de alojamiento. Por tanto, este paso no puede realizarse con ninguna cuenta y fue necesario usar una alternativa.

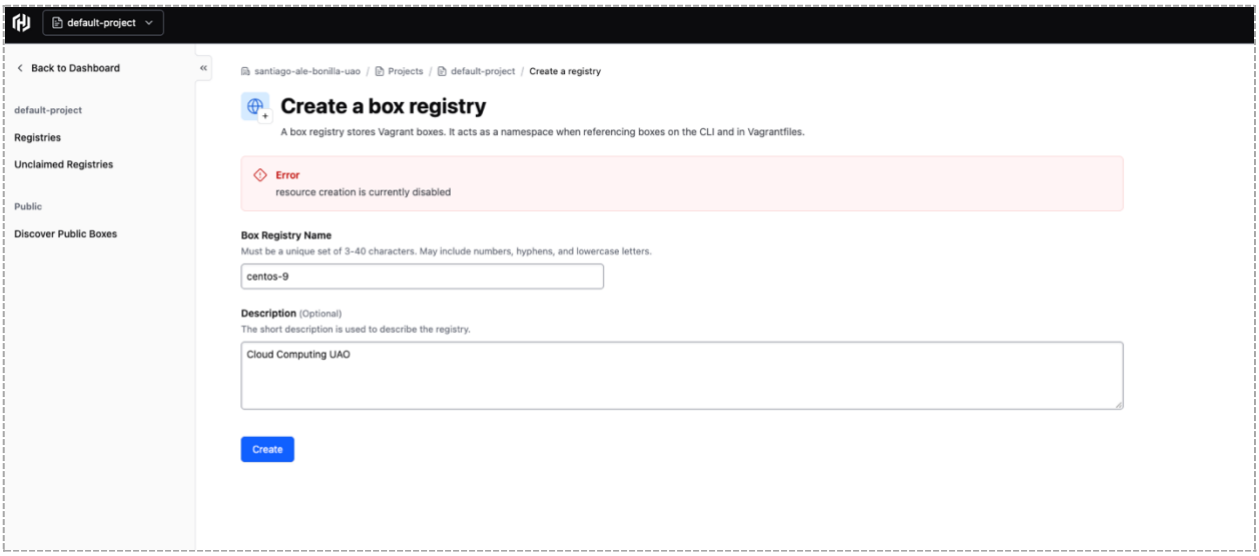


Figura 11. *error resource creation is currently disabled en HCP Vagrant Registry*

6. Análisis de resultados

Tabla 2. *Resumen de incidencias y soluciones*

Incidencia	Causa	Solución aplicada
VBOX_E_PLATFORM_ARCH_NOT_SUPPORTED	Box x86_64 en un anfitrión ARM64	Usar una box construida para arm64
bento/centos-stream-9 sin variante arm64	Box solo publicada para x86_64; CentOS 8 sin soporte desde 2021	Reemplazo por bento/centos-stream-9
resource creation is currently disabled	Descontinuación de HCP Vagrant Registry	Publicación en GitHub Releases con metadata.json

El laboratorio muestra que la portabilidad de un entorno definido como código depende tanto del archivo de configuración como de los artefactos que este referencia. Un Vagrantfile correcto no garantiza el despliegue si la imagen base no existe para la arquitectura del anfitrión, o si el servicio de distribución deja de operar. Este riesgo corresponde a la dependencia del proveedor (vendor lock-in) y a la continuidad del negocio, que el NIST señala como temas abiertos de la computación en la nube.

Por otro lado, la solución adoptada para la distribución conserva las propiedades esenciales del registro original (nombre, versionado, metadatos de provider y arquitectura, y verificación de integridad por checksum) sin depender de un servicio específico, lo que facilita migrarla a cualquier otro alojamiento.

7. Conclusiones

- Se aprovisionó correctamente un entorno de dos máquinas virtuales con Vagrant y VirtualBox, con conectividad por red privada entre cliente y servidor.
- La arquitectura del anfitrión determina qué imágenes pueden ejecutarse: en equipos Apple Silicon es necesario usar boxes ARM64, porque VirtualBox no emula x86_64.
- El reemplazo de CentOS 7 por bento/centos-stream-9 permitió mantener la compatibilidad con la guía sin depender de un sistema operativo sin soporte.
- La descontinuación de HCP Vagrant Registry evidencia el riesgo de depender de un único proveedor de servicios en la nube; distribuir la box con metadatos estándar en GitHub Releases mantiene el versionado y la verificación de integridad de forma portable.

8. Referencias

HashiCorp. (2024). Vagrant Cloud is moving to HCP. https://www.hashicorp.com/blog/vagrant-cloud-is-moving-to-hcp

HashiCorp. (2026). HCP Vagrant deprecation: important dates and migration guidance. https://www.hashicorp.com/en/blog/hcp-vagrant-deprecation-important-dates-and-migration-guidance

HashiCorp. (s. f.). Vagrant documentation. https://developer.hashicorp.com/vagrant/docs

Badger, L., Grance, T., Patt-Corner, R. y Voas, J. (2012). Cloud Computing Synopsis and Recommendations (NIST SP 800-146). National Institute of Standards and Technology.

Mell, P. y Grance, T. (2011). The NIST Definition of Cloud Computing (NIST SP 800-145). National Institute of Standards and Technology.

Oracle. (s. f.). Oracle VirtualBox User Guide. https://www.virtualbox.org/manual/

9. Directorios sincronizados en Vagrant

Los directorios sincronizados (*synced folders*) de Vagrant permiten compartir una carpeta del equipo anfitrión (host) con una o varias máquinas virtuales (guest). De esta forma, los archivos del proyecto pueden editarse desde el sistema operativo anfitrión, con sus propias herramientas de desarrollo, y ejecutarse dentro de la máquina virtual sin copiarlos manualmente. Este mecanismo separa el entorno de edición del entorno de ejecución, que es uno de los principios de los entornos de desarrollo reproducibles.

```
==> cliente: Attempting graceful shutdown of VM...
==> cliente: Checking if box 'bento/centos-stream-9' version '202510.26.0' is up to date...
==> cliente: Clearing any previously set forwarded ports...
==> cliente: Fixed port collision for 22 => 2222. Now on port 2249.
==> cliente: Clearing any previously set network interfaces...
==> cliente: Preparing network interfaces based on configuration...
    cliente: Adapter 1: nat
    cliente: Adapter 2: hostonly
==> cliente: Forwarding ports...
    cliente: 22 (guest) => 2249 (host) (adapter 1)
==> cliente: Booting VM...
==> cliente: Waiting for machine to boot. This may take a few minutes...
    cliente: SSH address: 127.0.0.1:2249
    cliente: SSH username: vagrant
    cliente: SSH auth method: private key
==> cliente: Machine booted and ready!
==> cliente: Checking for guest additions in VM...
==> cliente: Setting hostname...
==> cliente: Configuring and enabling network interfaces...
==> cliente: Exporting NFS shared folders...
==> cliente: Preparing to edit /etc/exports. Administrator privileges will be required...
==> cliente: Mounting NFS shared folders...
    cliente: /Users/santiagobonillaidrobo/vms/compartido => /home/vagrant/compartido
==> servidor: VM not created. Moving on...
santiagobonillaidrobo@Mac-mini-de-Santiago vms %
```

Figura 12. Los directorios sincronizados

```
logout
[vagrant@servidor ~]$ logout
santiagobonillaidrobo@Mac-mini-de-Santiago vms % vagrant ssh cliente -c "cat /home/vagrant/compartido/desde_mac.txt"
vagrant ssh servidor -c "cat /home/vagrant/compartido/desde_mac.txt"
Archivo creado desde el Mac
Archivo creado desde el Mac
santiagobonillaidrobo@Mac-mini-de-Santiago vms % vagrant ssh cliente

This system is built by the Bento project by Chef Software
More information can be found at https://github.com/chef/bento

Use of this system is acceptance of the OS vendor EULA and License Agreements.
Last login: Tue Nov 10 15:57:31 2026 from 10.0.2.2
[vagrant@cliente ~]$ pwd
/home/vagrant
[vagrant@cliente ~]$ ls
compartido
[vagrant@cliente ~]$ cd compartido/
[vagrant@cliente compartido]$ ls
desde_mac.txt
[vagrant@cliente compartido]$
```

Figura 13. Archivo compartido